Annex XII — Real-Time P2P Coordination Contexts: Governed Multi-Party Automation, Capabilities, and Evidence

Version 1.1 — 24 July 2026 · Part of the Optirando™ Canon · Status: Defensive Publication / Prior-Art Disclosure Language: English. Normative unless a passage is explicitly marked informative. Annex number provisional — assign per Canon sequence.

RFC-style keywords (MUST, MUST NOT, SHOULD, SHOULD NOT, MAY) are used where a normative requirement is intended.

XII.1 Executive Introduction

Real-world tasks frequently involve several people, devices, organizations, and automated systems. This annex specifies how multiple independent BEAP™ identities and sessions cooperate within one shared operational context — the **Coordination Context** — without merging identities, collapsing trust boundaries, or granting authority through membership. It extends, and does not modify, the bilateral WR Handshake architecture (Annex VII), the capability and information-flow governance of Annex X, and the finalizer execution model. Section XII.3 consolidates, as an explicit index, the disclosed combination of features of the overall system together with this extension. The model scales from two people assembling a cabinet to a multi-domain industrial intervention and to factory floors operating machine fleets from several manufacturers. The worked scenarios illustrating this range — consumer assembly, an enterprise decision network, and a multi-manufacturer factory floor — are provided as companion infographics (Figures XII-1–XII-3).

XII.2 Problem Statement

Existing multi-party tooling either distributes information without identity-bound authorization (chat, event buses) or gates actions on platform accounts rather than verifiable capabilities (workflow suites), and in both cases produces records rather than evidence. Safety-relevant, regulated, and cross-organizational tasks require: independent identities with independent trust relationships; automation across participant boundaries only along explicit, verifiable authorization paths; a common, verifiable task history with per-role visibility; and sealed, selectively disclosable evidence of decisions, executions, and completion. Commonly deployed tooling is not designed to provide this combination under one trust model; this annex specifies how WR/BEAP provides it.

XII.3 Disclosed Combination of Features — System Core and Coordination Extension

Disclosure statement. This section enumerates, in explicit form, the combination of features disclosed by this annex together with the base paper and its prior annexes. The purpose of this enumeration is verifiability and completeness of the disclosure — **not** an assertion of novelty or inventive step; no such assertion is made. Each feature is disclosed both individually and in

combination with any subset of the other features. This enumeration is a consolidated index of the disclosure, not its boundary: all mechanisms specified anywhere in the base paper and its annexes form part of the disclosure even where they are not repeated here. Feature identifiers F1–F28 are stable reference identifiers.

Essential combined disclosure. Within the enumeration below, the following combination is identified as the essential core of the disclosed system: **BEAP™, the Bidirectional Email Automation Protocol (F1–F5),** as the secure bidirectional automation transport over open infrastructure; **WR Code® (F8)** as the universal entry anchor; **the WR Handshake (F9)** — of which the Public variant is merely the publisher-facing admission situation — in which context and agreement are constituent parts of the agreement mechanism itself; **PoAE™ (Proof of Authoritative Execution, F12)** and **PoAC™ (Proof of Authoritative Capability, F13)** as the cryptographic proofs governing human authorization and capability acquisition; and **the WR Pointer with Augmented Overlay and context-aware adaptive WR menus (F21–F23)**, through which a verified publisher publicly declares deterministic AI automation capabilities that a user invokes by pointing: on screen via the cursor, or in the real world, where the pointer designates a physical object either by being captured in a camera image or by selecting objects within the camera or smartglasses view, with the help of AI vision capabilities; every consequence is individually authorized and evidenced. In combination, these elements provide governed, verifiable, bidirectional AI automation between independent parties over open internet infrastructure — a problem class that messaging and collaboration platforms are not designed to address. This core operates in conjunction with the repository transparency of F6–F7, which makes every declared automation externally inspectable and tamper-evident before it can run, and with the execution and evidence chain of F14–F17. Each element remains disclosed individually and in every sub-combination; consistent with the disclosure statement above, no assertion of novelty or inventive step is made.

A. Transport and content core (BEAP™)

- **F1.** The **Bidirectional Email Automation Protocol (BEAP™)**, by which automation offers, briefs, requests, state, and results are sent and received over open internet infrastructure — including ordinary email transport — in sealed, end-to-end-encrypted capsules with absolute recipient binding and unbypassable seal verification on every read path.
- **F2.** Text-purity: prompts, instructions, and automation content travel exclusively as text under allowlist-based positive construction — never as code. Nothing received over the transport is executable by construction.
- **F3.** Blob inertness: binary originals (attachments, PDFs, images) remain encrypted and inert; depackaging occurs only inside an isolated environment (microVM class); only normalized derivatives enter the system.
- **F4.** Normalization or deterministic verification of every risk-bearing content class: links, documents, images, and comparable artifacts are either normalized at ingest (format conversion, hash registration) or presented exclusively through a deterministic, controlled rendering pipeline with graded rendering assurance; images render only on hash match against their normalized repository version, including on live pages.
- **F5.** Channel provenance verification (SPF/DKIM/DMARC plus discovery records) emitting typed provenance records, surfaced uniformly and unsuppressibly to the user; verdicts

inform, never authorize (ratchet discipline). The entire ingest and provenance process is monitored by **WR Guard™**, which operates a variety of hot-swappable LoRA-class analysis adapters — among them **Scam Watchdog™**, identifying phishing risks and other risk-bearing patterns; adapter findings feed the provenance verdicts and remain informative under the same ratchet (enforcement per F19).

B. Public transparency and repository

- **F6.** A public repository in which publisher adapters, AI automation capabilities, pre-declared action chains, prompt profiles, LoRA adapters, normalized images, and all other publisher-offered instructions and artifacts are publicly listed, versioned, and hash-anchored via Merkle manifests — so that everything a publisher can invoke against a user is externally inspectable and tamper-evident before it can run.
- **F7.** Repository ingest discipline: restricted artifact classes (text artifacts, safetensors, images normalized into a converted format); automated pre-publication scanning; versioned manifest updates, never silent replacement.

C. Identity, agreement, and capability core

- **F8.** WR Code® as universal scan-based context anchor: an identifier and invitation — not authority.
- **F9.** The **WR Handshake** as the single agreement mechanism, in which **context and agreement are constituent parts of the handshake itself**: bidirectional per-direction capability declarations, pre-declared action chains, consent-class floors, declared role views, Merkle-manifested artifacts, and graded publisher attestation. The designations Internal, Public, and Cross-Device are UI-level identifiers of the admission situation, not separate mechanisms (§XII.4).
- **F10.** The two BEAP™ relationship classes: **pBEAP™** — the open path — persistent relationships granting delivery and preparation rights only, never execution authority, with unilateral revocation; the standing peer-to-peer connection enables automations and information exchange in near real time; and **qBEAP™** — post-quantum-ready encryption for confidential automation.
- **F11.** External Service Admission as a distinct governance class for non-WR counterparts (cloud AI APIs, external retrieval): signed deployment declarations, per-invocation assurance classes, mandatory pre-invocation transformations, and treatment of outbound queries as disclosures.
- **F12.** PoAE™ (**Proof of Authoritative Execution**): cryptographic proof of human authorization, risk-adaptively graded through consent classes C0–C4 (click, 2FA hardware key, passkey-protected, hardware attestation), bound to an intent hash, with escalation-only ratchet: consent floors may be raised, never lowered.
- **F13.** PoAC™ (**Proof of Authoritative Capability**): governance of all capability acquisition — skills, workflow recordings, sources, adapters, prompt profiles — under "propose, never silently apply", with provenance and mutation taxonomy.

D. Execution, boundary, and evidence core

- **F14.** Finalizer classes READ, TRANSMIT, CHANGE, PERSIST, EXECUTE under the lifecycle Validate → Execute → Witness → Seal → Anchor, inside a cryptographically governed execution authority (WR TED — explicitly not an operating system).

- **F15.** AI-as-router: models select among, parameterize, and branch within pre-declared execution chains; they never synthesize new execution paths at runtime.
- **F16.** WCR recordings (Workflow Context Recorder): recorded workflow chains — deterministic at every tool boundary, adaptive on path — declared through the WR Handshake and the repository (F6), rendered and replayed exclusively within the controlled rendering pipeline (F4), and terminating only in the deterministic endpoints of the finalizer classes (F14): a recording may prepare, propose, and route, but every consequence exits solely through a finalizer.
- **F17.** Evidence on three verification tiers — local, central, and external blockchain anchoring — for executions, consents, capability events, and boundary events.
- **F18.** Deterministic trust boundaries: any crossing of a trust boundary, in either direction, requires explicit cryptographic permission; no hidden path; honest-boundary discipline (residual risks stated, never obscured); locality-independent trust — internal devices, sandboxes, and field devices obtain capabilities through the same mechanisms as external parties.
- **F19.** WR Guard™ consuming WR Expert™ knowledge files: semantic pre-boundary analysis of outbound content (including prompts during typing), ten graded measures from warning to full block, operable advisory or enforced per finding class, with every finding and crossing logged as a boundary event record.
- **F20.** WR Graph™ as a four-dimensional knowledge graph (declarative, episodic, parametric, procedural) whose access is governed exclusively by cryptographically verifiable, requester-class-scoped capabilities.

E. Interaction surface

- **F21.** WR Pointer: a device- and interaction-adaptive layer — desktop AI cursor; on mobile, a real-world pointer via the camera **or** an on-screen cursor, depending on the level of interaction; smartglasses pointing; on-screen adaptive menus — acting as **trigger, never authority**; every consequence passes the capability → validation → consent → finalizer chain.
- **F22.** Augmented Overlay with **context-aware adaptive WR menus**: context-bound status, assistance, and offered AI automation capabilities rendered against the pointed-at object; offer-never-action. Routing is surfaced in place: the AI-as-router's selection among the pre-declared chains (F15) is displayed directly, context-bound, within the adaptive menu — the offered automations always reflect the current context.
- **F23.** Publisher-declared pointer automation: a verified website publicly declares — via its WR Public Handshake and the repository (F6) — deterministic AI automation capabilities for the WR Pointer and Augmented Overlay, optionally including publisher-supplied LoRA adapters; the user can equally add their own fine-tuned adapters, admitted under PoAC™ (F13) — such that pointing at the publisher's content can invoke exactly the declared and admitted automations, and nothing else.
- **F24.** Login-bound personalization: short-lived Login-Bound Capability Proofs bound to the handshake hash plus the authenticated site session (dual-possession binding); identifier-only slot discipline (account-scoped identifiers and typed declared operator inputs — no other slot class exists); logout as a cryptographic state change.

F. Multi-party coordination (this annex)

- **F25.** The Coordination Context: coordinates sessions; is not itself a session, an identity, or an authority; admission is membership — not authority (R1, R2).
- **F26.** Exactly three authorization path classes for consequential cross-participant actions; chain-head freshness binding; single-use, intent-hash-bound approvals; role-view-bounded disclosure (R3–R6).
- **F27.** Handshake Groups: per-asset WR Code contexts grouped under one WR Public Handshake per publisher; a declaration and admission scope, never a trust bridge; in multi-group contexts, per-publisher role views bounded to the publisher's own group by default (R11, §XII.13).
- **F28.** Exclusive proof origination: every proof object throughout F1–F27 is produced by BEAP and the WR Handshake; the architecture introduces no proof system, key hierarchy, or root of trust of its own beyond them (R10). Where declared, it **consumes** external verification anchors — blockchain anchoring for the external evidence tier (F17), email-authentication infrastructure for provenance verdicts (F5), hardware-attestation roots for the higher consent classes (F12), and site authentication for login binding (F24) — each admitted and declared, and none able to create, substitute, or elevate authority. Coordination outcomes are sealed as Task Completion Receipts and anchored per F17.

XII.4 Definitions

- **Coordination Context (CC):** a task-bound construct consisting of a signed Context Manifest, an append-only hash-chained context event log, and a set of admitted participants. **A Coordination Context coordinates sessions. It is not itself a session, an identity, or an authority.**
- **Context Manifest:** the versioned signed object constituting a CC (§XII.12.1). The hash of its initial version is the **CCID**.
- **Participant:** an identity admitted to a CC: a human operator, an organization, a policy engine, or a device holding consequential capabilities of its own. A device that acts only as an endpoint of an operator MUST NOT be admitted as a participant.
- **Role / Role View:** a manifest-declared function within the task, with a declared subset of event classes and payload fields visible to holders of that role.
- **Coordinator:** the participant currently holding the sequencing role (§XII.10). Ordering and admission-processing authority only.
- **Capability edge:** an admitted, verifiable authorization from one participant toward another (or toward task actions), in one of the three path classes of §XII.9.
- **Consequential action:** an action whose finalizer class is TRANSMIT, CHANGE, PERSIST, or EXECUTE, directed at another participant, their assets, or shared external state.
- **Handshake Group:** the set of contexts (e.g., per-machine WR Code contexts) governed by one WR Public Handshake. A declaration and admission scope, not a trust bridge (§XII.13).
- **WR Handshake:** The handshake mechanism previously referred to in this paper and its figures simply as "Handshake" is henceforth named the **WR Handshake**. Earlier figures and passages are not being globally revised; wherever they read "Handshake", "Public Handshake", or "Internal Handshake", the WR Handshake is meant. The designations *Internal*, *Public*, and *Cross-Device* are user-interface identifiers that help the user distinguish the admission situation — they are not separate cryptographic mechanisms. Together with

BEAP, the WR Handshake is the exclusive **origin** of cryptographic proof in this architecture; declared external verification anchors are consumed, never authority-creating (F28, R10).

XII.5 Object Encoding, Canonicalization, and Signature Scope

The following rules apply to every object defined in this annex:

- Every object carries a versioned type identifier of the form `wr.cc.<object>/<version>` (e.g., `wr.cc.approval/1`).
- Every object is serialized into a **canonical form** before hashing or signing: a deterministic encoding in which field order, string encoding (UTF-8), integer representation, and absence-of-field handling are fully specified, so that any two conformant implementations produce byte-identical serializations of the same object. Conformant schemes include canonical deterministic CBOR and JSON under a JCS-class canonicalization; the base paper's capsule encoding rules apply where the object travels inside BEAP capsules.
- Every signature covers the complete canonical form of the object **excluding only the signature field itself**, prefixed with a domain-separation tag equal to the object's type identifier. Signatures over partial objects are non-conformant.
- Every hash reference to another object (`*_ref`, `*_hash` fields) is a hash of that object's canonical form, using the hash function declared in the Context Manifest (SHA-256 class or stronger).
- All signing keys are participant keys established through BEAP sessions and WR Handshake admissions (R10). No object in this annex introduces its own key hierarchy.

XII.6 Group-Session Model

A CC links independent BEAP sessions; it does not create a super-session. - The CC MUST NOT hold capabilities, sign approvals, or execute actions. Every signature within a CC is a participant signature.

- Participant sessions MUST remain independently valid; termination of the CC MUST NOT terminate them, and vice versa (departure semantics in §XII.8).
- Multiple WR Public Handshakes from different publishers MAY coexist within one CC; each publisher participates under its own handshake declarations and role view. Their trust domains MUST NOT merge.
- A qBEAP participant and a pBEAP participant MAY cooperate in one CC; cross-domain authority exists only as explicit capability edges (§XII.9).
- Nested contexts SHOULD be represented as a child CC whose manifest references the parent CCID, with independently evaluated authorization; authority MUST NOT be inherited from parent to child.
- A CC is justified when the task involves three or more mutually gating participants, any cross-participant consequential automation, an evidence requirement, or multiple trust domains; otherwise bilateral sessions SHOULD be used. The model degrades to near-invisible simplicity in minimal cases.

Architecture rationale (informative). Three models were evaluated. **(A)** A root handshake with derived participant bindings gives simple admission but concentrates authority and availability in the anchor. **(B)** Fully independent linked sessions under a signed manifest preserve independence

but re-derive the coordination-authority question with worse ergonomics and fragmented auditability. **(C)** A pure capability-graph presentation gives exact authorization semantics but poor mainstream explainability. The specified model is the hybrid: A's admission topology (an anchored, versioned manifest), B's invariant (sessions never merge; the context can outlive its anchor per its survivability rule), and C's authorization semantics (all consequential automation requires an admitted capability edge; membership creates zero edges). The capability graph is the *representation* of §XII.9; rule R3 is its edge-existence *invariant* — the same statement at two levels of formality.

XII.7 Identity and Trust Model

- Organization, operator, device, and endpoint identities remain represented separately, as elsewhere in the Canon.
- **Pseudonymous participation** is a first-class mode: an admission MAY bind a role to a context-scoped key without identity disclosure. The manifest MUST declare, per role, whether identified or pseudonymous admission is acceptable; safety-critical approval roles SHOULD require identified admission.
- Participants MUST be able to see, per event, the *role* of the acting participant; identity disclosure beyond role is governed by the manifest and consent classes.
- A publisher coordinating a task MUST NOT receive participant identity information beyond what its declared role view specifies. Admission of additional participants MUST NOT require disclosure of their identities to the publisher unless the manifest declares this per role and the participant consents at admission.
- One operator MAY act through multiple devices (endpoints of one participant). Several people sharing one physical device MUST retain separate operator identities; per-action attribution follows the authenticated operator.
- Identity substitution is prevented by binding every event signature to the admitted participant key(s) in the Participant Admission Record; key changes require a re-admission event.
- **Trust summary.** No participant trusts another by virtue of membership. The Coordinator is trusted for ordering and liveness only; its misbehavior class is denial or forking — detectable via signed heads — never forgery of authority. A publisher is trusted exactly as far as its handshake declarations. Internal and external participants coexist because nothing in the context ever grants cross-domain authority; only explicit, scoped, revocable edges do.

XII.8 Admission and Participant Lifecycle

Lifecycle stages: creation → admission (including role assignment) → capability issuance → synchronization → proposal → approval → execution → suspension → revocation → departure → re-entry → completion → archival → evidence export → retention/deletion.

- **Creation:** any identity holding an admitted entry context (WR Public Handshake, internal workflow, WR Code–initiated flow) MAY instantiate a Context Manifest. Manifest amendments MUST be versioned and logged; silent replacement is prohibited.
- **Admission:** through any entry path admitted by the manifest: the participant's own handshake, a context-scoped invitation derived from an existing participant's session, local admission, or assisted capture (Annex IX classes). Every admission MUST produce a

Participant Admission Record counter-signed by the admitted participant. **Admission is membership — not authority.** Admission MUST NOT create any capability edge except role capabilities explicitly listed in the manifest and explicitly accepted by the participant at admission.

- **Late joining:** an admitted late joiner receives its role view of the chain from the head backward as authorized; it MUST NOT receive event classes outside its role view regardless of join time.
- **Suspension / revocation:** logged as events; revocation voids the participant's edges forward-looking; previously sealed evidence remains valid. Where group payload confidentiality is enabled (§XII.19), revocation MUST trigger key rotation for subsequent payloads.
- **Departure and re-entry:** departure is an event; the CC persists per its survivability rule. Re-entry is a fresh admission.
- **Initiator/anchor disappearance:** the manifest MUST declare a survivability rule (§XII.10, Coordinator succession) or a context freeze. Publisher-anchored consumer contexts SHOULD survive publisher unavailability for all locally executable steps.

XII.9 Capability and Delegation Model

The core invariant: **A Coordination Context MAY coordinate multiple BEAP sessions, but a consequential action by one participant toward another requires an admitted capability edge in exactly one of three path classes: (a) a pre-existing bilateral capability from an admitted WR Handshake; (b) a delegated capability with a verifiable delegation chain; (c) a role capability declared in the Context Manifest and accepted by the affected participant at admission. No other authorization path exists. Membership creates no edges.**

- Capabilities are issued per participant and scoped per role, per task, and per action class via the existing capability-token format extended with a `context_scope` field (§XII.12.6). Consent classes C0–C4 apply per action; the manifest declares the consent-class floor per action; ratchet discipline applies (escalation only).
- **Delegation:** a participant MAY delegate a capability it holds iff the capability is marked delegable. A delegated capability MUST be scope-narrowing in every dimension (actions, targets, validity; the consent floor may only rise); the delegation chain MUST be verifiable end-to-end; maximum chain depth is a manifest parameter with default 1; revocation of any chain element voids all descendants.
- **Proposal without execution:** any participant whose role includes proposal rights MAY propose an action it cannot execute; proposals are events, never executions. Proposed actions are drawn exclusively from pre-declared action chains (AI-as-router, F15) — never synthesized at runtime.
- **Approvals:** the manifest declares, per gated action class, the required approval set: single role, multiple roles, quorum predicate, and/or policy-engine decision. Approvals are Approval Records (§XII.12.4); a quorum is a manifest policy predicate evaluated over Approval Records, not a separate object. Policy conflicts resolve by declared precedence; Where an organizational policy and a manifest rule conflict, the stricter requirement prevails (ratchet).

- Capability updates on workflow-state change are manifest-declared transitions (e.g., a capability active only during a declared step), evaluated deterministically against the chain — never granted ad hoc at runtime.

XII.10 Shared-State and Event Model

- All context events travel as **Context Event Envelopes** (§XII.12.3): signed by the acting participant, sequenced by the Coordinator with (epoch, sequence, previous-hash), payload encrypted per role view, evidence objects embedded by reference, with a per-event Merkle structure for selective disclosure.
- **Authoritative state** is the deterministic fold of the validated chain. Snapshots are permitted as non-authoritative caches only. No separately signed "shared state" object exists; a signed snapshot would fork authority against its own ledger.
- **Consistency class:** task-specific sequenced consistency. Every consequential action MUST reference the chain head (hash + epoch) against which it was decided; the Coordinator MUST reject a consequential action whose referenced head has been superseded by a conflicting intervening event, as defined by the step's declared conflict set. This is the stale-client defense. General-purpose distributed consensus, CRDTs, and Byzantine agreement are out of scope (§XII.21).
- **Concurrency:** concurrent non-conflicting events interleave by Coordinator sequence; concurrent conflicting proposals are resolved by sequence order plus re-decision of the loser against the new head.
- **Partial failure:** unreachable participants accumulate their role-view events store-and-forward; steps whose approval sets cannot be satisfied block or fall to leased offline authority (§XII.17).
- **Coordinator misbehavior** is limited by construction to ordering faults (withholding, forking). Participants SHOULD cross-verify signed heads, making forks detectable; the Coordinator cannot forge events, approvals, or capabilities. Sequencing signatures order events; they never substitute for capability edges or approvals.
- **Coordinator succession.** Two conformant variants are specified: (i) a manifest-declared ordered succession list: on declared unavailability criteria, the next listed participant publishes a signed takeover event referencing the last verified head; (ii) election: participants holding manifest-declared approver roles issue Approval Records for a takeover proposal until the manifest's quorum predicate is met. Both variants are conformant; the manifest MUST declare which applies. In both, the successor gains ordering authority only.

XII.11 Decision, Approval, and Execution Flow

Proposal event (intent hash; action drawn only from pre-declared action chains, surfaced to participants directly and context-bound in the context-aware adaptive WR menus of the WR Pointer where that surface applies) → required Approval Records collected (each PoAE-gated at its consent class; policy-engine decisions as signed events) → manifest predicate satisfied → execution by the capable participant under the finalizer lifecycle Validate → Execute → Witness → Seal → Anchor → execution receipt event → state fold advances. An execution MUST reference the satisfied approval set and the head hash; approvals are single-use per intent hash and head.

XII.11a Context Disambiguation at the Interaction Surface

Where an operator holds multiple admitted handshakes or participates in multiple Coordination Contexts, the interaction surface MUST make the governing context identifiable at the moment of consequence. Cryptographic separation is already guaranteed at the object layer (ccid binding, `context_scope`, §XII.5, §XII.12.6); this section governs the ergonomics of intent, not authorization. Two display tiers apply:

Ambient context indicator. The Augmented Overlay SHOULD render the currently governing context — the task descriptor and, where applicable, the acting publisher's Handshake Group — as a context-bound overlay element alongside the context-aware adaptive WR menus (F22). It is an overlay element like any other: the operator MAY show or hide it under overlay configuration. It informs, never authorizes.

Consent-time context binding. For every consequential action, the canonical rendered intent — the object whose hash is the intent hash (§XII.12.4) — MUST include the ccid and the human-readable task descriptor of the governing context. The consent surface MUST display this context identification unsuppressibly; overlay configuration MUST NOT affect the consent surface. An Approval Record therefore always attests to a context the approver saw.

Indicator floor. The Context Manifest MAY declare, per role, that the ambient indicator is mandatory (non-hideable) for holders of that role. Ratchet discipline applies: indicator floors may be raised, never lowered at runtime.

No display authority. Rendering or hiding an indicator never creates, satisfies, or bypasses a capability edge (R9). An action directed at the wrong context remains invalid at the object layer by construction; this section addresses the residual risk of an operator forming intent against the wrong context while holding valid capabilities in both.

XII.12 Protocol Objects

This annex adds **five normative objects** and **two field extensions** to existing objects. All follow §XII.5. Rejected candidates and rationale: a separate Role Binding (folded into the admission record), a signed Shared Task State (state is derived, §XII.10), a Session Link Record (subsumed by admission), a Quorum Record (a predicate over Approval Records, not an object), and a separate Group Evidence Seal (covered by the Task Completion Receipt plus the chain).

XII.12.1 Context Manifest — `wr.cc.manifest/1`

Constitutes the CC. **Issuer:** the initiator, optionally under a WR Public Handshake (publisher co-signature where publisher-anchored). **Holder:** all participants. **Fields:** type; version; task descriptor; hash-function declaration; role table (role → view definition → role capabilities → identified/pseudonymous admissibility); entry-path policy and admission limits; per-action consent-class floors; per-gated-action approval predicates; sequencing rule and Coordinator succession variant (§XII.10); survivability rule; leased-offline-authority declarations (§XII.17); evidence policy (§XII.18); amendment rule; issuer signature(s). **Expiry/termination:** closed by a context-termination event; amendments are versioned manifest events, never replacements. The hash of version 1 is the CCID.

XII.12.2 Participant Admission Record — `wr.cc.admission/1`

Binds a participant to the CC. **Issuer:** the admission authority per manifest (Coordinator or delegated admitter), **counter-signed by the admitted participant** (acceptance). **Holder:** the participant and the context log. **Fields:** type; ccid; participant key(s); role(s); entry-path class (own handshake / derived invitation / local admission / assisted capture); pseudonymity flag; references to accepted role capabilities; admission epoch; both signatures. **Revocation:** a context revocation event referencing this record; key changes require re-admission.

XII.12.3 Context Event Envelope — `wr.cc.event/1`

Uniform signed carrier for all context events (state advance, proposal, approval reference, execution receipt reference, admission, revocation, takeover, completion). **Issuer:** the acting participant; sequenced by the Coordinator. **Fields:** type; ccid; epoch; sequence; previous-head hash; event class; role-view-encrypted payload; embedded evidence references (PoAE, consent receipts, execution receipts); per-event Merkle leaf structure for selective disclosure; actor signature; Coordinator sequencing signature. **Expiry:** none (append-only).

Worked instance — canonical fields are shown as annotated YAML for readability; the wire form is the canonical encoding of §XII.5:

```
object_type: "wr.cc.event/1"
ccid:        "sha256:4be1…"
epoch:       12
sequence:    347
prev_hash:   "sha256:c41d…"        # head at sequencing time
event_class: "proposal"
payload_enc:                        # per role view; keys per §XII.19
  view_technician:  "enc:…"
  view_supervisor:  "enc:…"
evidence_refs: []
merkle_root: "sha256:aa90…"        # leaves: one per field group, enabling
                                    # selective disclosure of single fields
actor_key:   "ed25519:pk:7Qc…"
actor_sig:   "ed25519:…"           # over canonical form excl. both sigs,
                                    # domain tag "wr.cc.event/1"
seq_sig:     "ed25519:…"           # Coordinator, over the same scope plus
                                    # (epoch, sequence, prev_hash)
```

XII.12.4 Approval Record — `wr.cc.approval/1`

Signed approval or veto of a proposed action. **Issuer:** the approving participant — a human via PoAE at the required consent class, or a policy engine via a signed policy decision. **Holder:** context log and proposer. **Fields:** type; ccid; proposal reference (hash of the proposal envelope); intent hash (the canonical intent the approver saw); head hash and epoch at decision time; approver key and role; verdict (approve/veto); consent class used; PoAE reference; issuance time; signature. **Staleness:** bound to the referenced head — stale by construction. **Single-use:** per intent hash and head (R5).

Worked instance:

```
object_type:   "wr.cc.approval/1"
ccid:          "sha256:4be1…"
proposal_ref:  "sha256:9f2c…"
intent_hash:   "sha256:77ab…"      # WYSIWYE: hash of the exact rendered intent
head_hash:     "sha256:c41d…"
epoch:         12
approver_key:  "ed25519:pk:Kd2…"
role:          "safety_officer"
verdict:       "approve"
consent_class: "C3"                 # 2FA hardware key
poae_ref:      "sha256:e0d9…"      # sealed PoAE record for this consent
issued_at:     "2026-07-23T14:02:11Z"
sig:           "ed25519:…"          # over canonical form excl. sig,
                                    # domain tag "wr.cc.approval/1"
```

XII.12.5 Task Completion Receipt — `wr.cc.completion/1`

Seals the final result. **Issuer:** the roles the manifest requires, jointly; sealed under the PERSIST finalizer. **Holder:** all participants; exportable. **Fields:** type; ccid; final head hash and epoch; outcome digest; signer set (role + key per signer); anchor references (tiers L/C/X); signatures. **Expiry:** none (terminal); superseded only by a versioned successor context. **Relationship to PoAE™ evidence:** the receipt neither replaces nor duplicates the per-action PoAE™ records and execution receipts — those remain the action-level evidence, embedded in the chain's event envelopes. The receipt is the task-level joint attestation of the required roles over the final head: a multi-party signature that must be created, not derived from the log, and the compact exportable object for third parties under selective disclosure. Its own sealing is a PERSIST finalizer action and therefore itself produces a PoAE™ record; because the receipt references the chain whose events carry the underlying execution receipts, a verifier can descend from the task-level attestation to every per-action proof.

XII.12.6 Field extensions to existing objects

- **Capability token:** two optional fields. `context_scope` — (ccid, role); the capability is valid only inside that context. `delegation_chain` — an ordered list of signed delegation statements, each naming delegator, delegate, narrowed scope, validity, and consent floor; subject to the §XII.9 narrowing and depth rules. This realizes delegated capabilities without a new object class.
- **Revocation:** the existing revocation mechanism gains a context scope (participant, role, capability, or whole-context revocation as event classes in the log). No separate revocation object format is introduced.

XII.13 Handshake Groups

A **Handshake Group** is the set of contexts — typically per-asset WR Code contexts — governed by one WR Public Handshake. The handshake carries, once, the publisher's attestation, capability declarations, pre-declared action chains, consent-class floors, and role-view templates for every context in its group. Scanning an asset's WR Code binds the operator's session to that asset's context **under the already-established handshake with that publisher** — no new handshake per asset. Asset identifiers enter as identifier-class slots, never payloads (Identifier-Only Slot Discipline). A group shares declarations, attestation, capability templates, and the versioned update path of its handshake; it does **not** share state — each context remains individually scoped.

- A Handshake Group is a declaration and admission scope, **not a trust bridge**. Membership of two assets in the same group creates no capability edges between their contexts; edges within a group arise only through the three path classes of §XII.9.
- A CC MAY be anchored under one group's handshake and admit multiple contexts of that group as participants (fleet tasks); the publisher's role view then covers exactly its own group's events.
- Multiple Handshake Groups MAY participate in one CC (§XII.6). Where they do, each publisher's role view MUST be bounded to events originating from or addressed to its own group, unless the manifest declares otherwise and the affected parties consent at admission; WR Guard™ enforces this bounding at each handshake's egress boundary. Only the CC initiator holds the cross-group view.
- Handshake Groups add grouping and admission structure — never proof machinery (R10).

XII.14 Transport Neutrality

Conformance is defined at the object layer — signatures, seals, recipient binding, chain integrity — not the transport layer. Direct peer transport, relay-assisted transport, brokered distribution, federated organizational nodes, centrally coordinated delivery of end-to-end-signed events, hybrid local/cloud, store-and-forward, and edge-local coordination are all conformant. Central *coordination* (ordering, delivery) MUST NOT be construed as central *authority* (authorization, evidence): no transport or coordination component can create, satisfy, or bypass a capability edge. The system is correctly described as a distributed, capability-governed multi-party automation fabric; "P2P" describes one conformant transport and the pBEAP relationship layer, not the architecture. Latency is a transport property: where a persistent peer connection (pBEAP class) exists, event distribution, approvals, and automation exchange operate in near real time; store-and-forward paths trade latency for reachability. Neither changes authorization or evidence semantics.

XII.15 Security Considerations

- **Unauthorized invitations:** invitations are context-scoped, manifest-policy-checked at admission, and produce counter-signed admission records; an invitation, like a WR Code, is an identifier and invitation — not authority.
- **Participant flooding:** manifest-declared admission limits and admitter roles; admissions are logged events, hence visible and revocable.
- **Malicious or compromised participants:** blast radius equals their capability edges. Isolation = revocation event + edge voiding + key rotation (§XII.19). They cannot forge

others' events (per-participant signatures) or approvals (PoAE-bound, intent-hash-bound, head-bound).

- **Replay:** intent hash + head hash + single-use approval binding + chain sequencing.
- **False approval or completion injection:** requires the private keys of the declared approver roles at the declared consent classes; neither a Coordinator nor any transport can inject either.
- **Revoked devices:** handled by existing device revocation; their admission records void.
- **Termination:** a context-termination event voids all context-scoped capabilities.
- **Live checks vs. leased authority:** the manifest MUST partition gated actions into live-check-required and leased-authority-eligible; runtime reclassification is prohibited.

XII.16 Privacy and Data Minimization

Role views bound disclosure by construction; pseudonymous admission bounds identity exposure; publishers receive only their declared view; selective disclosure via per-event Merkle proofs lets a participant prove its own participation, approvals, and the completion without revealing unrelated participants or events. The manifest MUST declare each role's view explicitly; undeclared event classes default to invisible.

XII.17 Offline and Degraded Operation

Store-and-forward applies to role-view events; edge-local Coordinators sustain site-local continuity. **Leased offline authority** — pre-issued, time-boxed, scope-boxed capabilities declared in the manifest — allows declared action classes to proceed during disconnection, with mandatory reconciliation events on reconnect. Actions outside leased scope block. On reconnect, each leased-authority execution is submitted as a reconciliation event referencing the pre-disconnect head; the Coordinator sequences it and evaluates it against the step's declared conflict set, yielding one of three declared verdict classes: *compatible* (folded into state), *superseded* (state unchanged; the physical-world effect is flagged for the declared remediation role), or *conflicting* (the context enters a declared exception step requiring the manifest's exception approval set). Verdict classes and the remediation role are manifest declarations; runtime invention of verdicts is prohibited.

XII.18 Evidence and Auditability

One context event chain (not N linked ledgers) is authoritative; participants additionally retain their own sealed copies of events they signed. Consequential events MUST embed their evidence objects (PoAE, consent receipts, execution receipts); non-consequential events (chat-class, telemetry below declared thresholds) MAY be logged without evidence objects — the manifest declares the evidence policy. Capability issuance and delegation are PoAC-visible events. The Task Completion Receipt seals the outcome under the PERSIST finalizer and anchors on tiers L/C/X.

XII.19 Group Confidentiality

Payload confidentiality is per role view. Two conformant realizations are disclosed; the manifest MUST declare which applies:

- **(i) Per-role pairwise encryption.** Each event payload section is encrypted to the current holders of the entitled role(s) using their BEAP session key material. Simple; no group key state; membership change requires no rotation protocol (departed members simply stop receiving); cost grows with role occupancy.
- **(ii) Group keying of the MLS class (RFC 9420 profile) under WR governance.** A continuous group key schedule per role view provides forward secrecy and post-compromise security across membership churn; the Coordinator operates the delivery-service function; **admission, removal, and every membership operation remain governed exclusively by this annex's admission and revocation objects** — the keying layer implements membership decisions, it never makes them, and it introduces no authority and no proof machinery of its own (R10).

Consumer-profile contexts SHOULD use (i); enterprise contexts with churn and long lifetimes SHOULD use (ii).

XII.20 Failure Modes

Coordinator loss (mitigated by succession, §XII.10; worst case an ordering halt, never authority loss); network partition (progress limited to satisfiable approval sets and leased authority); anchor or publisher disappearance (survivability rule; publisher-declared actions unavailable); key compromise (bounded by edges plus rotation; pre-compromise evidence unaffected); a forking Coordinator (detectable via cross-verified heads, not silently resolvable — human or organizational adjudication required); manifest misconfiguration (the model enforces declared policy; it cannot repair a badly declared policy).

XII.21 Non-Goals

This annex does not provide: perfect real-time consistency; general distributed consensus or Byzantine fault tolerance; elimination of central infrastructure; security by cryptography alone (governance, policy, and operational practice remain load-bearing); automatic legal compliance in any jurisdiction; a replacement for general workflow, collaboration, or message-bus systems; universal participant access to group data; any automation right arising from membership; mandatory peer-to-peer transport; and any assertion of novelty or inventive step relative to prior art — comparative statements in this annex describe the design scope of common tool categories, not the absence of prior art.

XII.22 Open Design Questions

1. Preference and profiling guidance between the two disclosed group-confidentiality realizations (§XII.19) beyond the stated recommendation; performance envelopes under churn.
2. Attestable sequencing: whether high-assurance classes require a hardware-attested Coordinator profile.
3. Canonical specification format for the deterministic state fold and its relationship to WCR step definitions.
4. Delegation ergonomics: whether depth-1 suffices for enterprise service chains, or scoped depth-2 with a mandatory intermediate organizational identity is needed.

5. Identity escrow for pseudonymous approvers where regulators require post-hoc attribution.
6. Cross-context capability references for nested or adjacent contexts (e.g., a warranty context referencing assembly evidence) without creating inheritance.
7. Consumer UX validation: the minimal consumer profile — one handshake, local admissions, one gated flow, one completion receipt — SHOULD be achievable with no more than two user decisions beyond the initial scan.

XII.23 Recommended Normative Rules (consolidated)

- **R1.** A Coordination Context coordinates sessions; it is not a session, an identity, or an authority; it holds no capabilities, signs nothing, executes nothing.
- **R2.** Admission is membership — not authority; admission creates no capability edges beyond manifest-declared, participant-accepted role capabilities.
- **R3.** Consequential actions (TRANSMIT/CHANGE/PERSIST/EXECUTE toward another participant) require an admitted capability edge in one of exactly three path classes: bilateral, delegated (scope-narrowing, chain-verifiable), or manifest-declared-and-accepted role capability.
- **R4.** Every consequential action references the chain head it was decided against; superseded heads with conflicting intervening events invalidate the action.
- **R5.** Approvals are participant-signed, PoAE-gated at declared consent classes, intent-hash- and head-bound, single-use.
- **R6.** Role views bound all disclosure; undeclared is invisible; publishers receive only their declared view.
- **R7.** Devices without consequential capabilities of their own are endpoints, not participants.
- **R8.** Manifest amendments are versioned and logged; runtime invention of authorization paths, consent downgrades, and silent policy changes are prohibited; ratchet discipline applies.
- **R9.** Conformance is object-layer; transport and coordination components can never create, satisfy, or bypass authorization.
- **R10.** All cryptographic proof within a Coordination Context is produced by BEAP and the WR Handshake — nothing else; the context and any Handshake Group introduce no independent proof system, key hierarchy, or root of trust.
- **R11.** A Handshake Group (the set of contexts governed by one WR Public Handshake) is a declaration and admission scope, not a trust bridge; group membership creates no capability edges, and in multi-group contexts each publisher's role view is bounded to its own group by default.
- **R12.** The governing Coordination Context is part of the canonical rendered intent of every consequential action and is displayed unsuppressibly at consent time; ambient context display in the Augmented Overlay is operator-configurable, subject to manifest-declared per-role indicator floors (escalation only).

XII.24 Conclusion

The Coordination Context extends WR/BEAP from bilateral relationships to multi-party tasks by adding exactly one construct — a governed, evidenced coordination layer — while changing nothing about how authority is created: capabilities, consent, finalizers, and evidence work

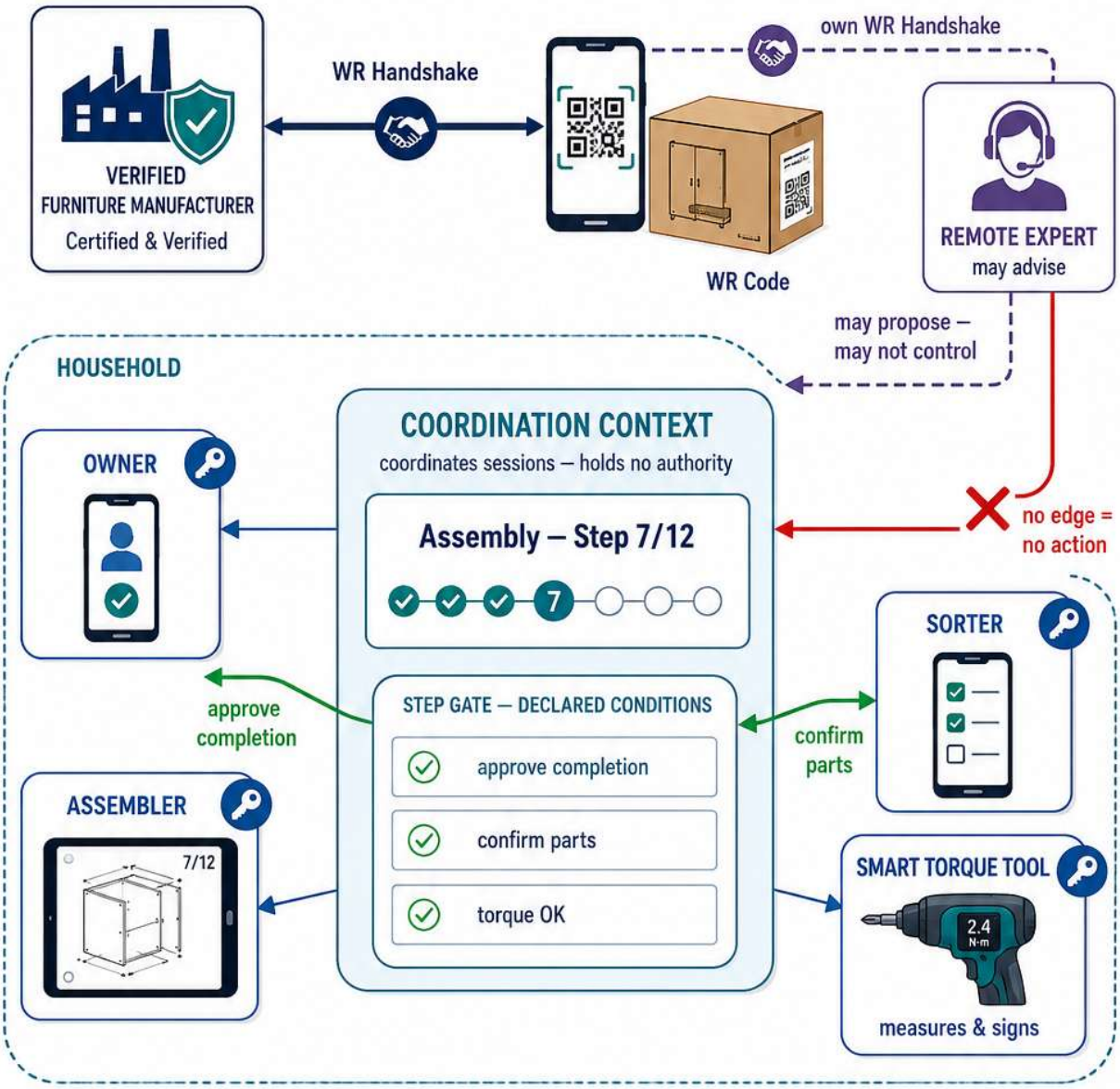
identically whether two parties interact or ten. Independent identities, sessions, policies, and automations cooperate on one verifiable history precisely because membership grants nothing and every consequence must still be individually, cryptographically earned.

XII.25 Version History

- **v1.0, 2026-07-23** — Initial version: Coordination Context model, disclosed combination of features (F1–F28) with essential combined disclosure, canonicalization and signature-scope rules, five normative objects with worked instances, capability-token field extensions, Handshake Groups (§XII.13), scenario figures as companion infographics (Figures XII-1–XII-3), disclosed alternatives for Coordinator succession, offline reconciliation, and group confidentiality, rules R1–R11.
- **XII.25 addition:** v1.1, 2026-07-24 — Added §XII.11a (context disambiguation at the interaction surface) and rule R12; no other sections modified.

One Task. Many Hands. Separate Keys.

A WR Coordination Context for everyday teamwork — cabinet assembly



Helpers join by scanning an invitation — the manufacturer never learns who they are.

The step advances only when the declared conditions are met.

Admission is membership — not authority.

TASK COMPLETION RECEIPT

Assembly completed — Step 12/12
Signed by the declared conditions in the coordination context.

2026-07-23 14:32:18 UTC

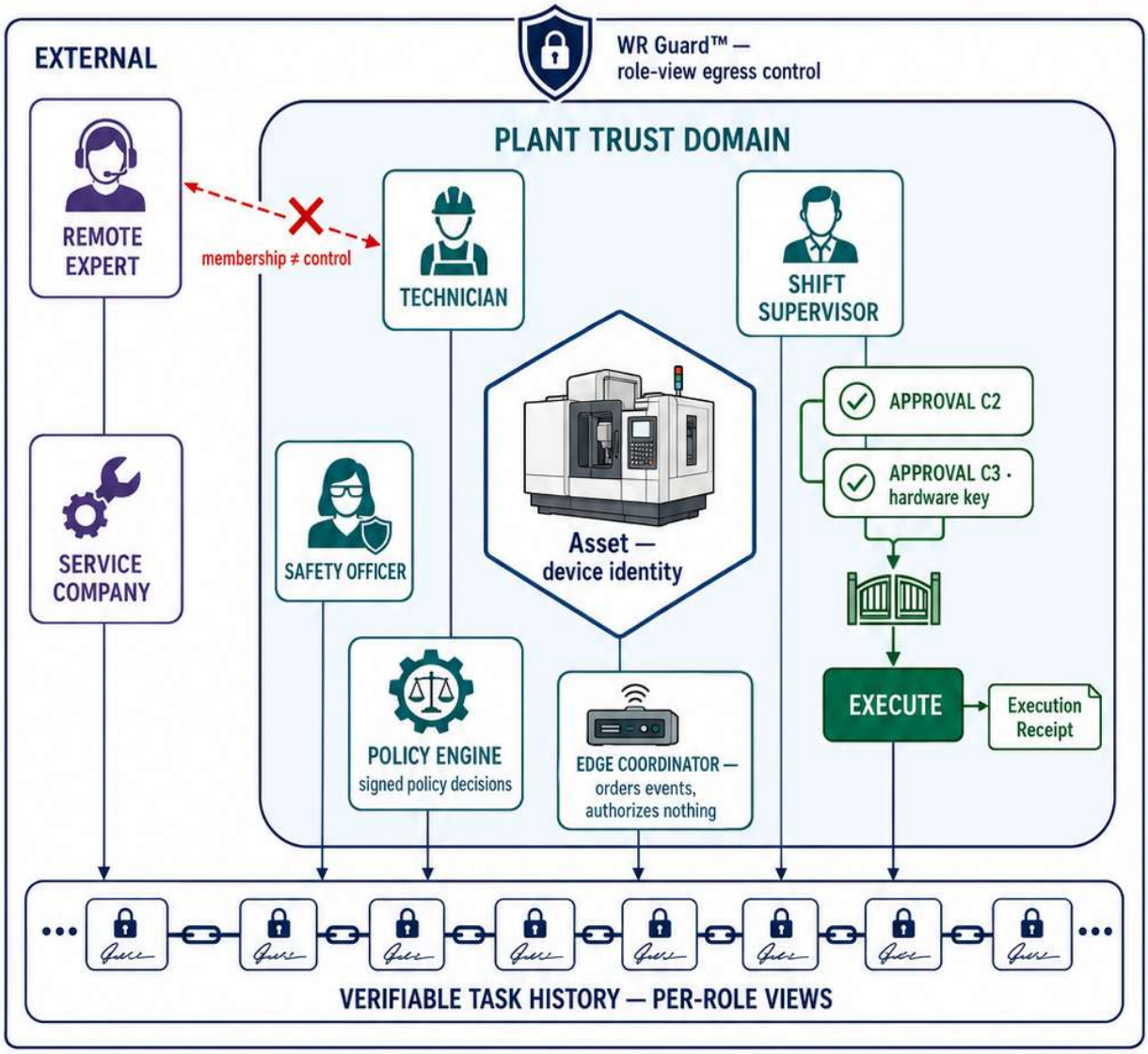
LOCAL Anchored · **CENTRAL** Anchored · **CHAIN** Anchored

Shared context does not mean shared identity or unrestricted mutual control.

FIGURE XII-2

Not Just Connected. Governed.

Capability-governed, verifiable distribution of decisions and automation across trust domains

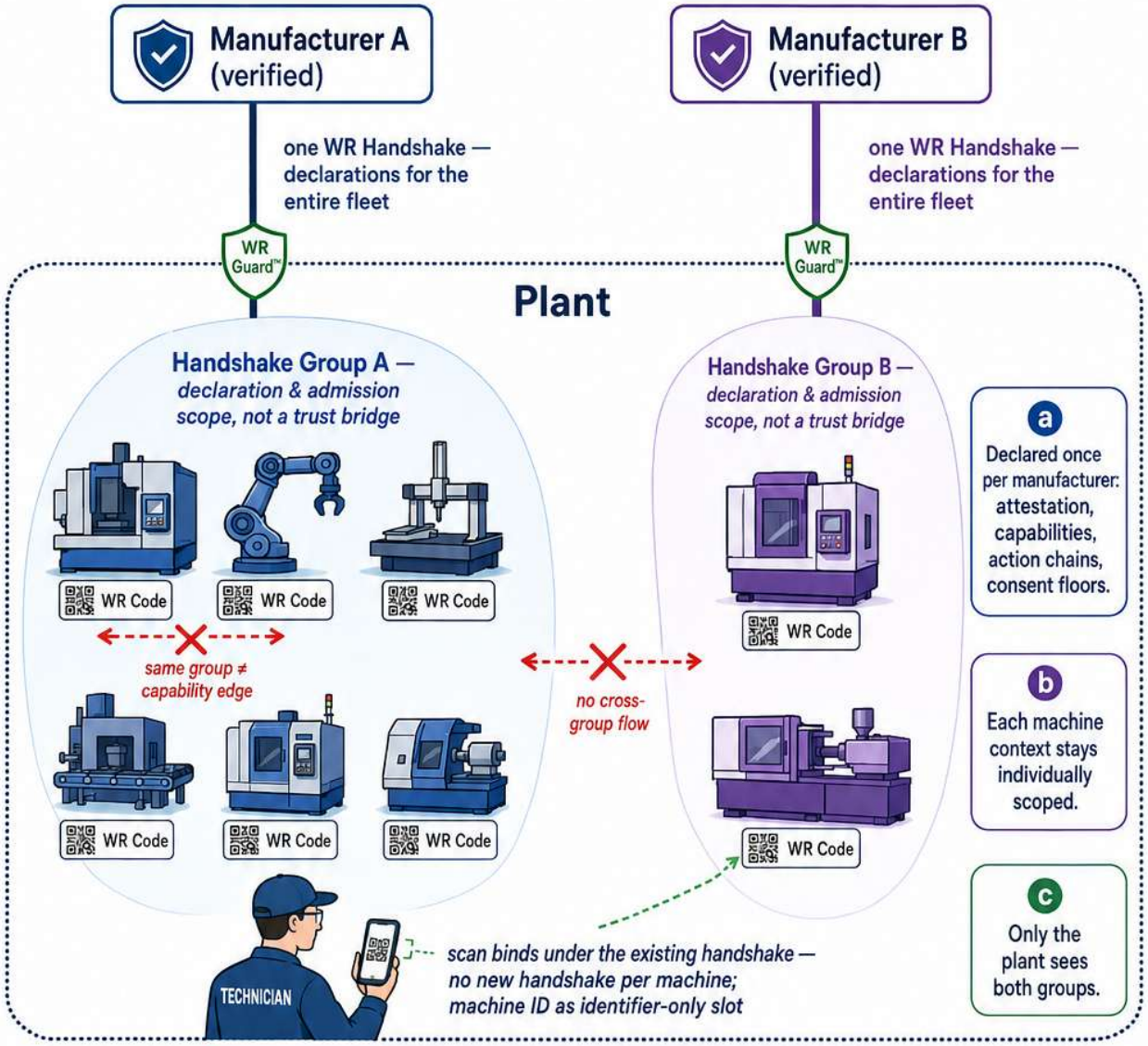


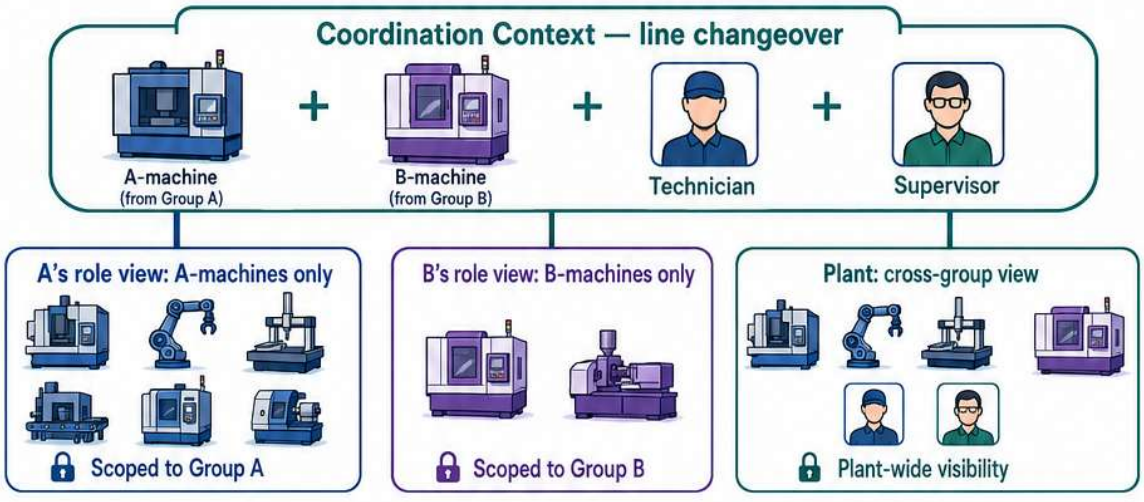
The innovation is not merely P2P transport. It is governed, verifiable, bidirectional distribution of decision and automation state.

Figure XII-3 — Handshake Groups on a Multi-Manufacturer Factory Floor

Two Manufacturers. One Floor. No Trust Bridge.

Per-machine WR Code contexts, grouped under one WR Handshake per manufacturer





A Handshake Group shares declarations — never state, never authority.

License Notice. This Annex forms an integral part of the Optirando™ specification and is licensed under the same terms as BEAP™, WR Desk™, and PoAE™.